`std::thread` constructor. But it's important to bear in mind that by default the arguments are *copied* into internal storage, where they can be accessed by the newly created thread of execution, even if the corresponding parameter in the function is expecting a reference. Here's a simple example:

```
void f(int i,std::string const& s);
std::thread t(f,3,"hello");
```

This creates a new thread of execution associated with `t`, which calls `f(3,"hello")`. Note that even though `f` takes a `std::string` as the second parameter, the string literal is passed as a `char const*` and converted to a `std::string` only in the context of the new thread. This is particularly important when the argument supplied is a pointer to an automatic variable, as follows:

```
void f(int i,std::string const& s);

void oops(int some_param)
{
    char buffer[1024];                    <-1
    sprintf(buffer, "%i",some_param);
    std::thread t(f,3,buffer);            <-2
    t.detach();
}
```

In this case, it's the pointer to the local variable `buffer` ❶ that's passed through to the new thread ❷, and there's a significant chance that the function `oops` will exit before the buffer has been converted to a `std::string` on the new thread, thus leading to undefined behavior. The solution is to cast to `std::string` *before* passing the buffer to the `std::thread` constructor: